

Backend Architecture

for the Offline-First Wound-Assessment Sync Component

Project J26-SE-330 — Member 4 Component

Version 2.0 — Consolidated from, and reconciled against, the four member proposal reports, including the ratified Recommendation Service Architecture v2.1

Drift/SQLite on the device · Apache Kafka (KRaft) as the event backbone · PostgreSQL as the system of record, with authenticated clinician access and a locally-linked patient record · Microsoft Agent Framework 1.0 orchestrating calls into Member 3's C#/.NET 10 evidence-grounded Recommendation Service

Designed so a wound assessment is never lost, never duplicated, never makes the clinician wait — and never reaches or leaves the system without a verified identity behind it.

1. Overview

This document sets out the full backend architecture for the offline-first sync component of Project J26-SE-330 (Member 4's responsibility). It is built from the four members' proposal reports and reconciled with Member 3's Recommendation Service Architecture v2.1, so that the two server-side components — synchronisation/orchestration and evidence-grounded recommendation — are described against one shared, ratified contract rather than a placeholder.

Five building blocks carry the design end to end:

- Drift/SQLite on the device — the offline queue, encrypted at rest, is the only way any assessment leaves the device.
- Apache Kafka (KRaft mode) as the event backbone — durable, ordered-per-wound, and the point after which an event is safe.
- PostgreSQL as the system of record — one schema per concern (clinical, messaging, audit, sync, rag), written through idempotent, version-checked, lock-aware transactions. The clinical schema carries device, wound and assessment data, a locally-linked patient record, and clinician identity/authentication.
- Microsoft Agent Framework 1.0 orchestrating calls into the Recommendation Service — a durable, checkpointed workflow that coordinates and records, but never itself decides clinical content.
- The Recommendation Service (Member 3) — C#/.NET 10, PostgreSQL + pgvector, with a deterministic admissibility gate, pre-computed retrieval, and a fully specified request/response contract.

The guiding constraint throughout is that the three properties in the title — never lost, never duplicated, never makes the clinician wait — are each the responsibility of a specific mechanism, not an aspiration. A fourth property runs alongside them: nothing is written or delivered without a verified clinician identity behind the device that sent it. Section 11 (Guarantees and Failure Handling) makes all four traceable, failure by failure.

2. End-to-End Architecture

2.1 Component view

Layer	Component	Responsibility
Device	Flutter UI + Drift/SQLite	Capture, offline queue, local read models, local sign-in cache (no secrets stored)
Gateway	Sync Gateway (ASP.NET Core, .NET 10)	Push/pull, clinician authentication (login/refresh/logout), figure proxy
Backbone	Apache Kafka (KRaft)	wound-events, wound-events.persisted, recommendations.ready, retry/DLQ topics
Persistence	Ingest persister, Outbox relay	Idempotent, version-checked, lock-guarded writes (§9)
Orchestration	Orchestrator (.NET, Agent Framework 1.0)	Checkpointed workflow calling the Recommendation Service; never decides clinical content
Recommendation	Recommendation Service (Member 3, C#/.NET 10)	Admissibility gate, retrieval, generation, contract v1.0 (§10)
Store	PostgreSQL + pgvector	clinical, messaging, audit, sync, rag schemas (§9)

2.2 The ten steps

A wound assessment's round trip, device to device, is ten steps. Authentication happens once per session, ahead of any push.

1. Authenticate. The clinician signs in once per session; the gateway issues a short-lived JWT plus a refresh token bound to the device and clinician_id (§7.3).
2. Push. The sync engine sends PENDING events in batches, bearing the JWT, whenever the gateway is reachable.
3. Accept. The gateway validates the JWT and each event, produces it to wound-events with acks=all, and returns a per-event result.
4. Consume. The ingest persister reads wound-events in its own consumer group.
5. Persist. One PostgreSQL transaction — schema-version-checked and lock-guarded — writes the assessment, upserts the linked patient record, writes provenance and an outbox row. The Kafka offset is committed only afterwards (§9.4).
6. Announce. The outbox relay publishes wound-events.persisted.
7. Orchestrate. The Agent Framework worker consumes it in a separate consumer group.
8. Recommend. It calls the Recommendation Service with wound analytics, clinical assessment and healing history only — no identifiers (§10).
9. Store. Recommendation, change-log entry, provenance and an outbox row are written atomically.
10. Deliver. The device pulls changes after its cursor, marks the record COMPLETE, and a SignalR nudge merely speeds this up.

2.3 Call flow

The device is involved only in steps 1, 2, 9 and 10 of §2.2 (authenticate, push, poll for changes, mark complete) — everything in between happens while the clinician is free to move on. Internally: Gateway → Kafka (wound-events) → Persister → PostgreSQL → Outbox → Kafka (wound-events.persisted) → Orchestrator → Recommendation Service → PostgreSQL → Outbox → Kafka (recommendations.ready) → Gateway (SignalR nudge, optional) → Device pull.

3. Decisions This Design Makes

Most of the architecture follows the four proposals directly. The rows below are the places where a choice had to be made, or where the wording in the proposals would not survive a hard question at the panel.

Topic	Decision	Why
Delivery guarantee	At-least-once on every hop plus idempotent handlers, described as effectively-once. Claim "0% duplicate records," not "exactly-once delivery."	Kafka's exactly-once semantics cover read-process-write inside Kafka. The device retry and the PostgreSQL write sit outside it. The mechanism is defensible; the phrase invites attack.
Who writes PostgreSQL	A persister consumes wound-events and writes PostgreSQL. The transactional outbox is used on the PostgreSQL-to-Kafka hop.	Keeps the proposed flow (Gateway, Kafka, PostgreSQL) and lets provenance store the real Kafka offset in the same transaction as the insert.
Head-of-line blocking	Partitioning, retry topics, a dead-letter topic, and separate consumer groups for persistence and orchestration.	Partitions alone still stall one partition behind a poison message. A slow AI call must never delay persistence.
Orchestrator boundary	Agent Framework (.NET) runs a workflow of typed executors and calls the Recommendation Service (C#, ASP.NET Core, .NET 10) over a versioned HTTP contract.	Matches Member 4's stack and Member 3's ratified stack; allows benchmarking with a stubbed service exactly as the methodology plans.

Topic	Decision	Why
Vector store	pgvector inside the same PostgreSQL server, separate rag schema, exact cosine search, no HNSW.	Member 3's decision log. Candidate sets must match the ones scored offline exactly; the corpus is only a few hundred rows.
Clinical assessment values	Three explicit strings — present, absent, not_recorded — end to end.	The Recommendation Service's safety gate depends on "unrecorded" never collapsing into "no." null or false would lose that.
Edits	Append-only revisions: assessmentId plus revision.	No merge conflicts, matches the regenerate-as-new-revision requirement, and gives the audit trail for free.
Returning advice to the device	Cursor-based pull is the source of truth. SignalR only nudges.	Works on flaky links. A dropped nudge loses nothing.
Identifiers	Pseudonymous patientRef on the sync path, stripped before the Recommendation Service call.	The Recommendation Service processes no patient identifiers.
Clinician identity	clinical schema owns clinician, clinician_credential and clinician_session tables; the gateway issues and verifies its own JWTs against them (§9.1, §7.3).	No external identity provider is in scope for the prototype; the panel needs a concrete, inspectable auth story.
Patient record	clinical.patient stores a minimal local record (patient_ref, facility_id, display_alias, an optional encrypted MRN reference); the sync path, Kafka payloads and the Recommendation Service boundary still see only patientRef.	Clinicians need to look up "whose wound is this" inside the app without re-deriving it from a pseudonym on every screen.
Schema versioning	Every schema (clinical, messaging, audit, sync, rag) is migrated through a numbered, checksummed migration ledger; a service refuses to start against an unexpected schema version (§9.2).	Five independently-owned schemas must fail loudly on mismatch, not corrupt silently.
Write concurrency	A transaction-scoped Postgres advisory lock keyed on assessment_id, taken ahead of every write to that assessment, on top of the existing unique-constraint idempotency (§9.3).	Retry-topic redelivery and horizontal scaling of the persister and orchestrator both mean two replicas can legitimately hold events for the same assessment at once; the lock makes ordering deterministic.

4. Deployable Units

Unit	Technology	Owner	Consumes	Produces	Scales by
Sync Gateway	ASP.NET Core, .NET 10	Member 4	Device HTTP (push/pull/auth); recommendations.ready for nudges	wound-events; HTTP responses; JWTs	Stateless replicas
Ingest persister	.NET Worker Service	Member 4	wound-events (group persister)	PostgreSQL rows, outbox rows	Consumers, up to partition count
Outbox relay	.NET hosted service	Member 4	PostgreSQL messaging.outbox	wound-events.persisted, recommendations.ready	Replicas using FOR UPDATE SKIP LOCKED
Orchestrator	.NET Worker Service with Microsoft Agent Framework	Member 4	wound-events.persisted (group orchestrator), retry topics	Recommendation rows, outbox rows	Consumers, plus concurrency per consumer
Recommendation service	C#, ASP.NET Core, .NET 10	Member 3	HTTP from the orchestrator	Cited recommendation or named refusal	Replicas
Kafka	KRaft mode, no ZooKeeper	Member 4	—	—	Partitions
PostgreSQL	PostgreSQL 16 with pgvector 0.8.x	Shared	—	—	Vertical, then read replica

The relay can live in the persister's process for the prototype. Keep it a separate class so it can be split out later.

Package versions for the Sync Gateway and the Recommendation Service are pinned in their own `apps/*/Directory.Packages.props`, not at the repository root: MSBuild applies the nearest such file up the tree, so a root file would govern both members' projects at once.

5. Wound Event Contract

Every boundary uses this one versioned JSON document. It is the only thing Members 1 to 3 need from the sync layer, and the only thing the sync layer needs from them.

```
{
  "schemaVersion": "1.0",
  "eventId": "0198f2c4-7b1e-7c3a-9d52-3e1f0a6b8c11",
  "assessmentId": "0198f2c4-7a90-7e02-8b1d-5c44e2f7a903",
  "revision": 1,
  "woundId": "0198f1aa-13d4-7f10-a2c8-9b70d3e5c611",
  "patientRef": "p-7f3a9c",
  "deviceId": "dev-a41c",
  "facilityId": "fac-001",
  "capturedAt": "2026-10-03T09:41:12+05:30",
  "analytics": {
    "areaMm2": 412.6,
    "colourRegions": [ { "cluster": 1, "percent": 61.2 }, { "cluster": 2, "percent": 27.9 } ],
    "fitzpatrickClass": "V",
    "pipeline": { "calibration": "2.1.0", "segmentation": "yolo11n-seg-0.4" }
  },
  "clinicalAssessment": {
    "pedalPulses": "not_recorded",
    "protectiveSensation": "absent"
  }
}
```

```
}  
}
```

- IDs are client-generated UUIDv7. eventId is the idempotency key for the whole pipeline. The server never invents an ID for an event.
- Revisions never update in place. Editing an assessment creates revision + 1 with the same assessmentId.
- Tri-state is strict. A missing clinical key fails validation. It is never read as not_recorded or as absent.
- Analytics only. No image bytes, no names, and a 16 KB cap per event.
- capturedAt is untrusted for ordering. Device clocks drift. Ordering uses revision and server timestamps.
- One schema, three enforcement points: a JSON Schema in a shared contracts/ folder, used by the Flutter app before enqueue, by the gateway before produce, and by CI contract tests.

6. Device: the Drift Queue

The offline queue is not a separate library. It is a table with a status column, written through one repository so Members 1 to 3 cannot bypass it. Every write to it commits before the UI shows "saved."

Table	Purpose	Key columns
wound_event_queue	The offline queue. One row per event, revisions are separate rows.	event_id PK, assessment_id, revision, payload_json, status, attempt_count, next_attempt_at, lease_expires_at, last_error, created_at, acked_at. Index on (status, next_attempt_at).
assessment_local	Read model so clinicians can review history offline. Latest revision per assessment.	assessment_id PK, latest_revision, summary_json
recommendation_local	Advice received from the server.	(assessment_id, revision) PK, payload_json, mode, received_at
figures_local	Cache of delivered guideline figures so they survive an app restart without a re-fetch.	figure_id PK, corpus_version, bytes, licence, cached_at
sync_state	Small key/value table.	server_cursor, last_success_at, device_id
auth_local	Cached clinician session state — never the raw refresh token, which lives in the platform keystore.	clinician_id, display_name, role, access_token_expires_at

Keep tokens out of Drift. Store the refresh token in the platform keystore, and encrypt the database file at rest with SQLCipher.

6.1 Record status

A record moves PENDING → IN_FLIGHT → ACCEPTED (or REJECTED) → ADVICE_DEFERRED → COMPLETE. ACCEPTED means durable in Kafka, which is what "synced" should mean on the device. COMPLETE is the "advice delivered" state. REJECTED rows stay visible in the UI and are never retried automatically.

6.2 Sync engine rules

- Triggers: connectivity change, app foreground, two seconds after an enqueue (debounced), pull-to-refresh, and a periodic background task. Android limits periodic background work to roughly every 15 minutes, so foreground triggers do most of the work.
- Reachability, not just connectivity: being on a network is not the same as the gateway being reachable. Probe GET /health with a short timeout before pushing.

- Single flight: one sync run at a time, guarded by a mutex and a database lease.
- Leases: rows sent are marked IN_FLIGHT with lease_expires_at about two minutes ahead. On app start, or when the lease lapses, they return to PENDING. This is what makes a killed app safe.
- Batching: up to 50 events or 256 KB per request, gzip-compressed, oldest first, with a result per event.
- Backoff: start at 2 seconds, double up to 5 minutes, full jitter, reset on success. Honour Retry-After on 429 and 503.
- UI budget: run the engine and all database work in a background isolate (Drift supports this), and let the UI watch Drift streams. That is how the 200 ms freeze limit is held.
- Auth: refresh the access token as the first step of a sync. If the refresh token has expired, pause and ask the clinician to sign in. The queue is untouched.

7. Sync Protocol

7.1 Push: device to server

```
POST /v1/sync/push
Authorization: Bearer <access token>
Content-Encoding: gzip

{ "deviceId": "dev-a41c", "batchId": "...", "events": [ { ...Wound Event... }, ... ] }

200 OK
{
  "batchId": "...",
  "results": [
    { "eventId": "0198...", "status": "ACCEPTED" },
    { "eventId": "0198...", "status": "DUPLICATE" },
    { "eventId": "0198...", "status": "REJECTED", "code": "PAYLOAD_TOO_LARGE" }
  ]
}
```

Outcome	Device does
ACCEPTED	Mark ACCEPTED. The event is durably in Kafka.
DUPLICATE	Treat exactly like ACCEPTED. This is how repeated reconnects end safely.
REJECTED (per event)	Mark REJECTED, show it, do not retry. The rest of the batch is unaffected.
401	Refresh the token, retry the batch.
413	Split the batch and retry.
429, 503, timeout, no response	Return rows to PENDING and back off. Safe because every event is idempotent.

The gateway checks the JWT's device_id claim against the body, and each event's facilityId against the token, so one device cannot submit as another.

7.2 Pull: server to device

```
GET /v1/sync/changes?cursor=8840&limit=100

200 OK
{
  "changes": [
    { "seq": 8841, "type": "RECOMMENDATION_READY",
      "assessmentId": "0198...", "revision": 1, "mode": "generated",
```

```

    "recommendation": { "...": "see RAG contract, §10.2" } }
  ],
  "nextCursor": 8841,
  "hasMore": false
}

```

Change types are PERSISTED, RECOMMENDATION_READY, ADVICE_DEFERRED and SUPERSEDED. The device upserts by (assessmentId, revision, type) and stores nextCursor in sync_state in the same transaction.

A cursor gotcha worth handling: database sequences can commit out of order. A reader that says "give me everything after 8840" can permanently miss row 8839 if it commits a moment later. The fix: the gateway also re-sends any change created in the last 60 seconds regardless of sequence number. Device upserts are idempotent, so the overlap is harmless.

7.3 Authentication: login, refresh, logout

A clinician signs in once per session; the device does not hold a long-lived password or API key.

```

POST /v1/auth/login
{ "username": "n.silva", "password": "...", "totp": "123456", "deviceId": "dev-a41c" }

200 OK
{
  "accessToken": "eyJ...",           // 15-minute JWT: sub, device_id, facility_id, role
  "refreshToken": "...",             // opaque, stored only as a hash server-side
  "expiresIn": 900
}

POST /v1/auth/refresh { "refreshToken": "..." } -> rotates both tokens
POST /v1/auth/logout  { "refreshToken": "..." } -> revokes the session row

```

- Five consecutive failed logins lock the credential for 15 minutes and write an audit.auth_audit row (§9.1, §12).
- A locked or expired credential returns 401 with a reason code; the offline queue is untouched — capture never needs a live token, only the sync step does.
- The refresh token is stored in the platform keystore on the device and, server-side, only as a SHA-256 hash in clinical.clinician_session — the raw token exists nowhere at rest.

8. Kafka Pipeline

8.1 Topics

Topic	Key	Partitions (demo / later)	Producer	Consumer group
wound-events	woundId	6 / 12	Gateway	persister
wound-events.persisted	woundId	6 / 12	Outbox relay	orchestrator
recommendations.ready	assessmentId	3 / 6	Outbox relay	gateway-notifier
wound-events.retry.30s, .retry.5m	woundId	3	Any consumer on failure	orchestrator (delayed)
wound-events.dlq	woundId	1	Any consumer, after retries	Manual replay tool

Keying by woundId keeps all revisions of one wound in order on one partition while spreading different wounds across partitions. Six partitions allow the 1, 2, 3 and 6 consumer scaling experiment the scalability requirement implies.

8.2 Client settings that matter

- Producer: `acks=all`, `enable.idempotence=true`. Headers carry event-id, schema-version, device-id and the W3C traceparent.
- Consumers: `enable.auto.commit=false`. Commit only after the database transaction commits.
- Broker: a single node for the demo, with replication factor 1. Do not claim durability under broker loss from that setup. A production layout is three brokers, replication factor 3, `min.insync.replicas=2`.
- Serialisation: JSON plus JSON Schema with a version header. A schema registry with Avro or Protobuf is a reasonable "future work" line.

8.3 Retries without blocking

On failure the consumer copies the message to the next retry topic and commits the original offset, so the partition keeps moving. The retry consumer reads each message's timestamp and pauses its partition until the delay has elapsed. This, not partitioning by itself, is what actually removes head-of-line blocking. The persister and the orchestrator are separate consumer groups on separate topics, so a five-second Recommendation Service call cannot delay a 5 ms database insert.

9. PostgreSQL

PostgreSQL is the system of record, one schema per concern, written through idempotent transactions.

Schema	Contents	Written by
clinical	device, patient, clinician, clinician_credential, clinician_session, wound, wound_assessment, recommendation	Gateway (auth, patient upsert), persister (assessments), orchestrator (recommendations)
messaging	outbox, inbox (processed-message table)	Persister, orchestrator; relay marks published
audit	provenance (append-only); auth_audit (login/refresh/logout, lockouts)	Gateway, persister, orchestrator
sync	change_log with an identity server_seq, a device cursor table, schema_migrations	Persister, orchestrator, gateway (cursor); migration tool
rag	guideline_chunks, query_vectors, rerank_scores, embedding_canaries, derivation_thresholds, figures, figure_links, verified_tables, corpus_versions, requests/retrievals/protocols/audit — see §10.5	Recommendation Service only

9.1 Clinical schema: identity, authentication and the patient record

Clinician identity, credentials and sessions, and a local patient record, live in clinical alongside device, wound and wound_assessment, so a single role-scoped schema answers "who submitted this, and whose wound is it" without granting the Recommendation Service any visibility into it — that service's database role holds no grant on clinical at all (§10.5).

Table	Purpose	Key columns
clinical.clinician	One row per registered clinical user of the Flutter app.	clinician_id PK, username (unique), full_name, role (nurse wound_specialist admin), facility_id FK, active, created_at
clinical.clinician_credential	Authentication secret, held separately from the identity row so it can be rotated, locked or purged without touching the profile.	clinician_id PK/FK, password_hash (Argon2id), password_salt, mfa_secret_encrypted (nullable), failed_attempts, locked_until, password_updated_at
clinical.clinician_session	Server-side record of issued refresh tokens, one per device/session, so a device can be revoked individually.	session_id PK, clinician_id FK, device_id FK, refresh_token_hash (SHA-256, never the raw token), issued_at, expires_at, revoked_at, last_seen_at
clinical.patient	Minimal local patient record so the app can show "whose wound is this". Not a full MPI — the hospital's own system remains the source of truth for identity.	patient_id PK, patient_ref UNIQUE (the same pseudonym used on the sync path), facility_id FK, display_alias (clinician-entered, e.g. initials or bed number), mrn_reference_encrypted (optional, pgcrypto column, facility opt-in), created_at, updated_at

Design intent: the sync path, Kafka payloads and the Recommendation Service boundary still see only patientRef. clinical.patient exists purely so the gateway and the Flutter UI can resolve a friendly label for clinicians already

authenticated and already inside the facility's access boundary; it is never read by the persister's Kafka-facing insert, never sent to Member 3's service, and `mrn_reference_encrypted` is column-level encrypted with `pgcrypto` so a database dump alone does not disclose it.

9.2 Schema versioning

Five schemas evolve independently. Every migration is a numbered, checksummed SQL file, tracked centrally so a mismatch between the code a service was built from and the database it connects to is a startup error, not a silent corruption.

- `sync.schema_migrations(schema_name, version, checksum, applied_at, applied_by)` — one row per applied migration, across all five schemas, ordered by version within each schema.
- Migrations live under `db/migrations/<schema>/NNNN_description.sql` (Appendix A) and are applied by a small migration runner (`dbup` or `EF Core Migrations`), run once per deploy, before any service starts.
- Every service reads its own expected schema version from its build (an embedded constant, bumped in the same PR as the migration) and calls `SELECT version FROM sync.schema_migrations WHERE schema_name = $1 ORDER BY version DESC LIMIT 1` on startup. A mismatch fails the health check rather than letting a service write against a schema it doesn't understand.
- Migrations are additive-first: a column is added nullable, backfilled, then made `NOT NULL` in a later migration, so the previous service version keeps working during a rolling deploy.
- A migration file, once applied to any environment, is never edited — a correction is a new migration, mirroring the append-only convention already used for provenance and the rag corpus versions.

9.3 Transaction locking

Kafka's per-wound partitioning keeps most writes single-threaded per wound, but retry-topic redelivery and horizontal scaling of the ingest persister (up to the partition count, §4) both mean two persister replicas can legitimately hold events for the same `assessment_id` at once. Locking makes the outcome deterministic instead of relying on timing.

Mechanism	Where it is used	What it prevents
Postgres advisory lock, <code>pg_advisory_xact_lock(hashtext(assessment_id))</code>	First statement inside the persister transaction (§9.4), and inside the orchestrator's <code>PersistResult</code> step	Two replicas racing to insert/supersede revisions of the same assessment; transaction-scoped, released automatically on commit or rollback
Unique constraint + <code>ON CONFLICT DO NOTHING</code>	<code>wound_assessment(event_id),</code> <code>wound_assessment(assessment_id,</code> <code>revision),</code> <code>recommendation(assessment_id,</code> <code>revision)</code>	A duplicate delivery becoming a duplicate row, even if the advisory lock were somehow bypassed
Isolation <code>READ COMMITTED</code> (default); <code>statement_timeout = 5s, lock_timeout = 2s</code>	All persister and orchestrator connections	A stuck lock blocking the partition indefinitely; a timeout fails the transaction and lets Kafka redeliver instead of hanging
<code>SELECT ... FOR UPDATE</code> on the current-latest row	<code>SupersedeCheck</code> executor, deciding whether an incoming revision is superseded by one already stored	A stale revision overwriting a newer one under concurrent orchestrator instances

Mechanism	Where it is used	What it prevents
FOR UPDATE SKIP LOCKED	Outbox relay's poll of messaging.outbox	Two relay replicas double-publishing the same outbox row

Lock ordering is fixed to avoid deadlocks: the advisory lock on `assessment_id` is always acquired before any row lock on `wound_assessment`, `patient`, or `recommendation`, and never the reverse, in every executor that touches more than one of them.

9.4 The persister transaction

The schema version is checked once at Worker startup (§9.2). Each transaction then takes the advisory lock, upserts the linked patient record, and writes the assessment, provenance, outbox and change-log rows atomically.

```
BEGIN;
SET LOCAL lock_timeout = '2s';
SET LOCAL statement_timeout = '5s';

-- 1. serialize all writers for this assessment
SELECT pg_advisory_xact_lock(hashtext(:assessment_id));

-- 2. upsert the local patient record (facility-scoped, PII-minimal)
INSERT INTO clinical.patient (patient_ref, facility_id, display_alias)
VALUES (:patient_ref, :facility_id, :display_alias)
ON CONFLICT (patient_ref) DO UPDATE SET updated_at = now();

-- 3. insert the assessment (idempotency key: event_id)
INSERT INTO clinical.wound_assessment
(event_id, assessment_id, revision, wound_id, patient_ref, device_id,
captured_at, received_at, analytics, clinical_assessment, status,
kafka_topic, kafka_partition, kafka_offset)
VALUES (...)
ON CONFLICT (event_id) DO NOTHING;

-- if 1 row inserted:
INSERT INTO audit.provenance (event_id, stage, outcome, device_id, kafka_ref, trace_id)
VALUES (... , 'PERSISTED', 'OK', ...);
INSERT INTO messaging.outbox (topic, msg_key, payload) VALUES ('wound-events.persisted', ...);
INSERT INTO sync.change_log (device_id, change_type, assessment_id, revision)
VALUES (... , 'PERSISTED', ...);

-- if 0 rows inserted (a duplicate was absorbed):
INSERT INTO audit.provenance (event_id, stage, outcome, ...)
VALUES (... , 'DEDUPPLICATED', 'OK', ...);

COMMIT;
-- only now: commit the Kafka offset
```

Persistence data — what this transaction writes, and why:

Row(s) written	Retention / notes
clinical.patient (upsert, at most one row per patient_ref)	Kept for the life of the facility relationship; display_alias only, no free-text PII unless mrn_reference_encrypted is enabled
clinical.wound_assessment (insert, append-only via revision)	Kept indefinitely; this is the clinical record
audit.provenance (PERSISTED or DEDUPPLICATED)	Append-only, insert-only role grants (REVOKE UPDATE, DELETE); this is the evidence the effectively-once claim is measured from (§13)

Row(s) written	Retention / notes
messaging.outbox (one row)	Deleted or marked published by the relay after a successful Kafka publish; short-lived by design
sync.change_log (one row, identity server_seq)	Kept until every device cursor has passed it, then eligible for archival

The DEDUPLICATED rows are worth keeping — they are the evidence that the mechanism actually absorbed repeats during the jitter tests. Give each service its own database role with least privilege: the rag role sees only the rag schema; only the gateway, persister and orchestrator can insert into audit.provenance and nobody can update or delete from it; only the gateway can read clinical.clinician_credential, and even the gateway never selects password_hash outside the login/refresh code path. Use Npgsql with Dapper or EF Core; the persister's ON CONFLICT statement is easier to keep exact in plain SQL.

9.5 Entity-relationship model

wound_assessment has unique constraints on event_id and on (assessment_id, revision); recommendation has a unique constraint on (assessment_id, revision). Core relationships:

- device (1) — submits —> (many) wound_assessment
- wound (1) — has —> (many) wound_assessment; wound.patient_ref resolves to (1) patient
- wound_assessment (1) — advice per revision —> (many) recommendation; — traced by —> (many) provenance; — announces —> change_log, outbox
- clinician (1) — issues —> (many) clinician_session; clinician (1) — has —> (1) clinician_credential
- wound_assessment.device_id and the authenticating clinician_session together answer "who submitted this, from which device, under which login" for the audit trail, without adding a clinician_id column to wound_assessment itself — the session-to-device mapping is enough, and keeps the clinical write path decoupled from whoever is currently logged in on that device.

10. Agent Framework and the Recommendation Service

The orchestrator's workflow is a graph of typed executors joined by conditional edges, run by the Agent Framework .NET SDK, which checkpoints progress at superstep boundaries so a crashed worker can resume. Keep the clinical decision out of the orchestrator: Member 3's safety properties (the admissibility gate, named refusals, citation verification) are deterministic and live in the Recommendation Service. The orchestrator coordinates, retries and records; it does not decide clinical content. That answers "where is the agent?": the framework provides the durable workflow, and any language model sits behind the Recommendation Service boundary where its output is verified.

10.1 Executors

Executor	Does	On failure
InboxCheck	Skips a message already processed by this consumer (unique on consumer name plus eventId).	Database error: retry topic.
SupersedeCheck	If a higher revision of the same assessment already exists, marks this one SUPERSEDED. Handles retry topics reordering revisions.	None; terminal.
BuildContext	Loads earlier revisions and earlier assessments of the same wound for healing history. Removes patientRef, deviceId and facilityId.	Retry topic.
CallRag	HTTP call to the Recommendation Service with a timeout, bounded retries and a circuit breaker.	After retries: ADVICE_DEFERRED, retry topic.
ValidateResponse	Checks the response schema and that every recommendation carries a citation. Does not re-judge clinical content.	Retry topic, then DLQ.
PersistResult	One transaction, advisory-locked: recommendation, change log, provenance, outbox, inbox marker.	Rolls back; message redelivered.

This is also why PostgreSQL comes before orchestration in the flow: the healing history the Recommendation Service needs for its stalled-healing escalation rule is data the persister has just stored.

10.2 RAG Contract v1.0

Endpoint: POST /v1/recommendations. No identifiers cross this boundary: caseId is the assessment UUID, never patientRef, deviceId or facilityId — the BuildContext executor strips them beforehand.

Request

Field	Notes
contractVersion, caseId, revision	caseId = assessment UUID; revision matches the sync-path integer
capturedAt, receivedAt	Used for the elapsed-time and stalled-healing (BPS 15) calculation
woundAnalytics	areaMm2, colourRegions, pipeline (calibration/segmentation versions). No fitzpatrickClass — data minimisation
clinicalAssessment	ulcerLocation, protectiveSensation, pedalPulses, probeToBone, infectionGrade, woundBedLabels. Every field's value set includes not_recorded; a missing key fails validation (422)

Field	Notes
healingHistory	Prior captures with areaMm2 and pipeline version, for the percent-area-reduction trajectory

Response

Field	Notes
mode	"generated" or "extractive" (no language model reachable — still a 200)
retrievalMode	"hybrid" (normal) or "lexical" (a vector lookup missed; logged as a defect)
corpusVersion	e.g. iwgd-2023.r1 — stored in provenance so an answer traces to a frozen guideline snapshot
sections, citations	Cited protocol text with [Sn] tags resolving to a chunk hash; an unresolved tag rejects the whole response server-side
withheld, nextBestAssessment	Named refusals with the missing field and blocked count — never silently dropped
escalation	triggered, rule, par, elapsedWeeks, clock — the stalled-healing flag consumed by the device UI
figures	Tier B/A references only, served via GET /v1/figures/{corpusVersion}/{figureId} (§10.4)
audit	Every generation attempt, the embedding/rerank model versions, and gate decisions

10.3 Behaviour rules

Rule	Orchestrator behaviour
Idempotency, keyed on (caseId, revision)	A repeat delivers the stored response unchanged; do not re-issue a call for a (caseId, revision) already checkpointed past PersistResult
422 (validation) or 409 (idempotency conflict)	Do not retry. Route to the DLQ; these indicate a contract or ordering bug, not a transient failure
Degraded answers (mode: extractive, or a full withheld list)	Treat as success. Store and deliver exactly like a generated answer
503	Retrieval itself is impossible (database down or no frozen corpus). Retry via retry.30s / retry.5m, then DLQ, exactly as for a network timeout
Deadlines	Up to two generation attempts at 20 s each on the service side; the orchestrator's CallRag timeout is 60 s (provisional), above the internal 40 s budget plus retrieval
Comparability failure ("measurement pipeline changed")	Deliver the response as-is; the trajectory section already reports it withheld

10.4 Figures

Figures are served at GET /v1/figures/{corpusVersion}/{figureId} and proxied through the Sync Gateway so the device never talks to the Recommendation Service directly. The Drift schema's figures_local table (§6) caches a delivered figure so it survives the next app restart without a re-fetch.

10.5 rag schema

Table	Contents
guideline_chunks	Chunk text, chunk_type, req_prerequisites, mentioned_fields, polarity, GRADE, embedding vector(256), tsvector, corpus_version
query_vectors	Pre-computed embeddings for every closed-set query template × slot-value combination
rerank_scores	Pre-computed rerank score per (query key, candidate chunk); the request path only joins against this table
embedding_canaries	Five fixed texts and vectors per corpus version, used to detect a silent embedding-model change
derivation_thresholds	Every runtime clinical threshold, with its quoted source text and pdf_index
figures, figure_links	Figure bytes, licence, attribution, tier, and verbatim cross-reference evidence
verified_tables	Human-verified guideline tables
corpus_versions	Manifest hash, embedding/reranker model + instruction hash, K, PostgreSQL/pgvector versions, frozen_at
requests, retrievals, protocols, audit	Idempotency store; retrieval and gate-decision rows; composed protocols; append-only generation audit

This schema is owned entirely by the Recommendation Service's own database role, which holds no grant on clinical, messaging, audit or sync — the same least-privilege boundary enforced for every other service role (§9.4).

11. Guarantees and Failure Handling

Hop	Delivery	What makes it safe
Device to gateway	At-least-once	Retry until ACK. eventId is the idempotency key.
Gateway to Kafka	At-least-once	Idempotent producer, acks=all. A re-sent batch after a crash can still produce twice, so downstream dedupes.
Kafka to PostgreSQL	At-least-once	Offset committed after the transaction. Unique event_id with ON CONFLICT DO NOTHING.
Outbox to Kafka	At-least-once	Relay may republish after a crash. Consumers use the inbox table.
Orchestrator to PostgreSQL	At-least-once	Unique recommendation per (assessment_id, revision) plus inbox.
Overall	Effectively-once	Duplicates can occur in transit; they can never become duplicate records.

Failure	What happens
App killed mid-sync	Lease expires, rows return to PENDING, next sync resends, server answers DUPLICATE or ACCEPTED.
Response lost after the server accepted	Same as above. Retrying is always safe.
Kafka unreachable	Gateway returns 503 with Retry-After. Nothing was acknowledged, so nothing is lost. The device keeps the queue.
Persister crashes after commit, before offset commit	Message is redelivered. ON CONFLICT absorbs it and a DEDUPLICATED provenance row is written.
Persister crashes mid-transaction	Transaction rolls back and the offset was never committed. Redelivered.
Poison message	Two retry hops, then the DLQ. The partition keeps moving.
Relay crashes after publishing	Row is republished. Downstream inbox drops the repeat.
Orchestrator crashes mid-workflow	Offset not committed, message redelivered, workflow reruns. The unique recommendation constraint stops a second write.
Recommendation Service slow or down	Timeout, bounded retries, circuit breaker. Record becomes ADVICE_DEFERRED; retry topics try again.
Recommendation Service returns 503	Retry via retry.30s / retry.5m, then DLQ (§10.3).
Recommendation Service returns 422/409	Not retried; routed straight to the DLQ for manual review (§10.3).
Oversized or invalid event	Rejected individually at the gateway. Others in the batch proceed.
Clinician credential locked (5 failed logins)	401 with a locked reason code; the offline queue is untouched (§7.3).
Schema version mismatch at service startup	Service fails its health check and does not start (§9.2).
Advisory lock held longer than lock_timeout	Transaction aborts; offset not committed; message redelivered and retried, safe because every write is idempotent (§9.3).

Hop	Delivery	What makes it safe
100 devices reconnect together		Batches are bounded, backoff has jitter, and Kafka absorbs the burst. Consumer lag rises then drains.

12. Security and Privacy

- Transport: TLS 1.3 everywhere, including gateway to Kafka and gateway to PostgreSQL in any non-local deployment.
- Identity: short-lived JWT access tokens (around 15 minutes) with claims sub, device_id, facility_id and role, issued by the gateway against clinical.clinician and clinical.clinician_credential. The gateway rejects events whose deviceId or facilityId do not match the claims. Devices are registered and can be revoked.
- Passwords are hashed with Argon2id, never reversible encryption; the salt is per-credential and never reused.
- Refresh tokens are stored server-side only as a SHA-256 hash (clinical.clinician_session.refresh_token_hash); the raw token exists only in transit and in the platform keystore on the device.
- Offline use: capture never needs the network or a live token. Only the sync step does.
- At rest on the device: SQLCipher-encrypted Drift file, key held in the Android Keystore through secure storage; auth_local (\$6) never holds the raw refresh token.
- clinical.patient.mrn_reference_encrypted, where a facility opts in, is column-encrypted with pgcrypto; the encryption key is held outside the database, in the same secrets store as the JWT signing key.
- Data minimisation: only analytics cross the wire to the Recommendation Service. No images, no names, no clinician or patient identity tables. This is enforced by the schema (unknown fields rejected), not just by convention.
- Access: pull responses are scoped by facility. Database roles are per service, with audit.provenance and audit.auth_audit insert-only. Only the gateway's role can read clinical.clinician_credential; the persister, orchestrator and Recommendation Service roles have no grant on it at all.
- audit.auth_audit records every login attempt (success or failure), lockout and logout, append-only — never used to authenticate, only to detect and rate-limit abuse.
- Stretch: hash-chain the provenance rows (each row stores a hash of the previous) to make tampering detectable.

Provenance stages recorded per event: GATEWAY_ACCEPTED, PERSISTED (or DEDUPLICATED), ORCHESTRATION_STARTED, RAG_RETURNED, RECOMMENDATION_STORED, DELIVERED. Each row carries device ID, Kafka topic/partition/offset where relevant, outcome, trace ID and timestamp, plus the RAG audit reference (model and corpus version).

13. Observability and Evaluation Harness

Build the measurement in from the start. Propagate the W3C traceparent header from the device through the gateway, into Kafka message headers, and out of the consumers, using OpenTelemetry. Export metrics to Prometheus and view them in Grafana.

Metric	How to measure it
Consumer lag	Kafka consumer-group lag for orchestrator while 100 events are processed. Compare with the REST baseline's total request time.
Duplication rate	Count rows sharing an eventId, and also count DEDUPLICATED provenance rows.

Metric	How to measure it
Sync latency	From device enqueue to persisted_at, with device-clock offset controlled for (see below).
Mobile responsiveness	Flutter frame timings during sync. Fail the run if any frame exceeds 200 ms.
Auditability completeness	SQL: events whose set of provenance stages is missing an expected stage, divided by all events.
Authentication health	Login failure rate, lockouts per day, and average session lifetime, from audit.auth_audit.

The duplication metric is 0% by construction — with a unique constraint on event_id, PostgreSQL cannot hold a duplicate. Report two more numbers: how many duplicate deliveries the pipeline absorbed (the DEDUPLICATED count), and an ablation run with the constraint and inbox switched off, plus a REST baseline without idempotency, showing what duplication looks like without the mechanism. Device clocks are unreliable; run the latency experiments with a device simulator on the same host or a synchronised clock, and say so in the method.

13.1 Harness pieces

- Device simulator: a script that creates synthetic Wound Events, keeps a real queue with the same states and lease logic, and speaks the real push and pull protocol, including login. Run 1 to 100 of them.
- Network faults: Linux tc netem, or Toxiproxy in front of the gateway for latency, drops and disconnect cycles.
- Fault injection: kill a consumer, send an oversized event, make one event slow through the Recommendation Service stub, replay a batch twice, lock out a credential.
- Baseline endpoint: POST /v1/baseline/assessments writes PostgreSQL and calls the same Recommendation Service stub inside the request. Same database, same stub, so the only variable is the architecture.

14. Deployment and Build Order

For the prototype, run everything with Docker Compose on one machine or one cloud VM: Kafka (KRaft, single broker), PostgreSQL with pgvector, gateway, persister, orchestrator, Recommendation Service, OpenTelemetry collector, Prometheus, Grafana, Toxiproxy, and a Kafka UI for debugging.

14.1 Build order, aligned with the integration stages

1. Shared schema. JSON Schema, generated fixtures, contract tests. Done when Members 1 to 3 can validate their output against it.
2. Clinical schema, auth and patient tables, and the schema_migrations ledger. Done when a clinician can register, log in, and the login is auditable end to end.
3. Drift queue. Repository, statuses, leases, encrypted database. Done when 1,000 offline saves survive an app kill.
4. Sync engine against a fake gateway. Backoff, batching, per-event results, login/refresh. Done when a flaky fake never causes a lost or stuck row.
5. Gateway and Kafka. Auth, validation, produce with acks=all. Done when a batch returns correct per-event results.
6. Persister and PostgreSQL. Advisory lock, idempotent insert, patient upsert, outbox, provenance. Done when replaying a topic twice leaves the row count unchanged.
7. Orchestrator with a Recommendation Service stub. Workflow, inbox, retries, DLQ. Done when a killed worker resumes and no recommendation is stored twice.
8. Real Recommendation Service, pull path, harness. Swap the stub for Member 3's service, add change-log pull, then run the benchmark suite against the REST baseline.

14.2 Scope tiers

Tier	Items
Core	Drift queue and leases, clinician login, push protocol with per-event results, gateway, Kafka with retry topics and DLQ, persister with advisory locking, outbox, orchestrator, change-log pull, provenance, REST baseline, netem tests.
Should	OpenTelemetry tracing across Kafka, SQLCipher, the scaling experiment across 1, 2, 3 and 6 consumers, the duplicate-mechanism ablation, MFA on clinician login.
Stretch	SignalR nudges, hash-chained audit, DLQ replay tool, schema registry, three-broker cluster.

15. Open Items to Settle with the Team

- Clinical form fields. The final tri-state field list, and where the shared schema lives.
- Patient identifier policy. Confirm a pseudonymous patientRef, plus the facility-opt-in encrypted MRN reference, is acceptable for the demo and the ethics story.
- Guarantee wording. Agree "effectively-once, zero duplicate records" as the phrase used across all four reports and the panel slides.
- Latency method. Agree the clock approach for the sync-latency experiments.
- Recommendation Service feasibility spike. The C#/.NET 10 decision is gated on it; confirm timeline.
- woundBedLabels. To be agreed with Member 1 (colourRegions are currently unlabelled clusters).
- MFA scope. Whether TOTP is mandatory for all clinician roles or optional for the prototype.
- Data-protection review. Whether the local patient record and the login audit trail need sign-off under the same PDPA review Member 3 has raised for the Recommendation Service (their §8.6).

Appendix A. Backend Repository Structure

One monorepo holds all four backend-facing units (Sync Gateway, Ingest persister, Outbox relay, Orchestrator) plus Member 3's Recommendation Service and offline ingestion pipeline, the shared contracts, database migrations, and the evaluation harness. Each app is an independently deployable unit (§4) but shares the contracts/ folder so a schema change is a single PR reviewed by whoever it affects.

```
J26-SE-330-backend/
├── README.md
├── global.json                # pinned .NET 10 SDK version
├── Directory.Build.props      # shared MSBuild settings
├── docker-compose.yml         # Kafka, Postgres, all services, OTEL, Grafana
├── docker-compose.override.yml # local dev overrides
├── .editorconfig
├── .github/workflows/
│   ├── ci.yml                 # build, unit tests, contract tests
│   ├── migration-check.yml    # applies db/migrations/ to a scratch DB
│   └── a1-parity.yml          # Member 3: C# vs Python A1 oracle diff
├── contracts/                 # the ONE schema every member builds against
│   ├── wound-event.schema.json # §5
│   ├── sync-changes.schema.json # §7.2 pull payload
│   ├── auth.schema.json       # §7.3 login/refresh/logout
│   ├── rag-request.schema.json # §10.2
│   ├── rag-response.schema.json # §10.2
│   ├── CHANGELOG.md
│   └── tests/                  # CI contract tests, all producers
├── db/
│   ├── migrations/
│   │   ├── clinical/
│   │   │   ├── 0001_create_clinical_schema.sql
│   │   │   ├── 0002_device_wound_assessment.sql
│   │   │   ├── 0003_recommendation.sql
│   │   │   ├── 0004_clinician_and_credential.sql
│   │   │   ├── 0005_clinician_session.sql
│   │   │   └── 0006_patient.sql
│   │   ├── messaging/0001_outbox_inbox.sql
│   │   ├── audit/
│   │   │   ├── 0001_provenance.sql
│   │   │   └── 0002_auth_audit.sql
│   │   ├── sync/
│   │   │   ├── 0001_change_log_cursor.sql
│   │   │   └── 0002_schema_migrations.sql # §9.2
│   │   └── rag/ # owned by Member 3, mirrored for CI
│   │       ├── 0001_guideline_chunks.sql
│   │       ├── 0002_query_vectors_rerank_scores.sql
│   │       ├── 0003_figures.sql
│   │       └── 0004_corpus_versions_audit.sql
│   ├── seed/ # local-dev only, never run in prod
│   └── README.md # migration-authoring rules, §9.2
├── apps/
│   ├── sync-gateway/ # ASP.NET Core, .NET 10 – Member 4
│   │   ├── SyncGateway.csproj
│   │   ├── Directory.Packages.props # pinned here, not at repo root
│   │   ├── Program.cs
│   │   ├── appsettings.json
│   │   └── Endpoints/
│   │       ├── PushEndpoint.cs # POST /v1/sync/push
│   │       ├── PullEndpoint.cs # GET /v1/sync/changes
│   │       ├── AuthEndpoints.cs # /v1/auth/login|refresh|logout
│   │       └── FiguresProxyEndpoint.cs # GET /v1/figures/...
```

- Auth/
 - JwtTokenService.cs
 - PasswordHasher.cs # Argon2id
 - ClinicianCredentialStore.cs
 - SessionStore.cs
 - Validation/ # JSON Schema, vs contracts/
 - Kafka/WoundEventProducer.cs
 - SchemaVersionGuard.cs # startup check, §9.2
- ingest-persister/ # .NET Worker Service – Member 4
 - IngestPersister.csproj
 - Worker.cs
 - Consumers/WoundEventsConsumer.cs
 - Persistence/
 - PersisterTransaction.cs # §9.3/§9.4
 - PatientUpsertRepository.cs
 - AdvisoryLock.cs
 - WoundAssessmentRepository.cs
 - outbox-relay/ # .NET hosted service – Member 4
 - OutboxRelay.csproj
 - RelayWorker.cs # FOR UPDATE SKIP LOCKED poll
 - orchestrator/ # .NET Worker Service, Agent Fwk – Member 4
 - Orchestrator.csproj
 - Workflow/OrchestratorGraph.cs # §10.1, typed executors
 - Executors/
 - InboxCheckExecutor.cs
 - SupersedeCheckExecutor.cs # SELECT ... FOR UPDATE
 - BuildContextExecutor.cs # strips identifiers
 - CallRagExecutor.cs # §10.2, 60s timeout
 - ValidateResponseExecutor.cs
 - PersistResultExecutor.cs # advisory lock + atomic write
 - recommendation-service/ # ASP.NET Core, .NET 10 – Member 3
 - RecommendationService.csproj
 - Directory.Packages.props
 - Program.cs
 - Endpoints/
 - RecommendationsEndpoint.cs # POST /v1/recommendations
 - FiguresEndpoint.cs # GET /v1/figures/...
 - WoundRag.Core/ # pure C#: gate, predicates, composer
 - WoundRag.Shared/ # hashing, normalisation, tokenizer
 - WoundRag.Infrastructure/ # SQL retrieval, gate, rerank lookups
 - Generation/
 - IGenerationAdapter.cs
 - QwenAdapter.cs
 - DeepSeekAdapter.cs
 - ingestion-pipeline/ # C# console app, Phase A – Member 3
 - IngestionPipeline.csproj
 - Steps/ # A1 Recon ... A9 Verify+Freeze
 - A1_Recon.cs
 - A2_Extract.cs
 - A3_Chunk.cs
 - A4_Annotate.cs
 - A5_Figures.cs
 - A6_Embed.cs
 - A7_Load.cs
 - A8_CandidatesAndRerank.cs
 - A9_VerifyAndFreeze.cs
 - infra/
 - kafka/create-topics.sh # wound-events, .persisted, .ready, dlq

```
|— postgres/init/          # extensions (pgvector in rag schema)
|— otel-collector/otel-collector-config.yaml
|— prometheus/prometheus.yml
|— grafana/dashboards/
|— toxiproxy/toxiproxy.json # netem-style fault injection
|
|— tests/
|   |— contract-tests/      # validates every producer against contracts/
|   |— integration/        # one wound assessment, device to device
|   |— device-simulator/   # §13.1 harness piece
|   |— fault-injection/    # kill-consumer, oversized-event, replay
|
|— docs/architecture/
|   |— backend-architecture.docx # this document
|   |— recommendation-service-v2.1.pdf # Member 3's document (reference)
```

Note: Directory.Packages.props is pinned inside apps/sync-gateway/ and apps/recommendation-service/, not at the repository root, because MSBuild applies the nearest such file up the tree and a root file would wrongly govern every .NET project in the solution.